# Ai for Bharat

## 1. Brief About the Idea

## Idea Name - AI Truth Resolution Engine

The **AI Truth Resolution Engine** is an AI system designed to handle a growing, real world failure of the digital age **contradictory information**. Instead of generating answers like a chatbot or listing links like a search engine, the system **ingests multiple conflicting sources**, detects contradictions, evaluates source reliability and timeliness, applies user-specific context, and outputs a **single best available answer** accompanied by **confidence scoring and transparent reasoning**.

The core value is not “answering questions” but **resolving disagreement** in information where multiple answers exist and users must make decisions.

## What the System Is

- A **decision-support intelligence layer**
- Focused on **arbitration, not generation**
- Built to work **above existing information systems**, not replace them

It answers the question:

> “I see different answers — which one should I trust, and why?”

## What the System Is NOT

To be very clear:

- Not a chatbot
- Not a search engine
- Not a fact-checking website
- Not a news summarizer
- Not a hallucination-heavy “assistant”

Those systems **produce or retrieve information**.

This system **judges between information**.

---

## Core Problem It Solves

Modern users face:

- Multiple official sources
- Frequent policy updates
- Platform-specific narratives
- Social amplification of half-truths

The problem is **not lack of data**, but **lack of arbitration**.

Humans are forced to:

- manually compare sources
- guess which is authoritative
- trust virality over validity
- delay decisions due to uncertainty

This system automates that arbitration **responsibly**.

---

## Core Capabilities (High-Level)

1. **Multi-source ingestion**
    - Government notices
    - Official documents
    - News articles
    - Platform content (where permitted)
2. **Contradiction detection**
    - Identifies when two or more sources disagree on:
        - facts

    - rules
    - eligibility
    - timelines
3. **Credibility & recency evaluation**
    - Source type
    - Authority level
    - Publication/update time
    - Domain relevance
4. **Contextual resolution**
    - User location
    - Category (student, business, citizen)
    - Applicable rules vs non-applicable ones
5. **Explainable output**
    - Selected answer
    - Confidence score (not certainty)
    - Reasoned justification

---

# Example

**User Query:**

“Is this exam eligibility rule applicable to me?”

**System Behavior:**

- Finds 4 conflicting interpretations:
    - old notification
    - coaching blog
    - YouTube explanation
    - latest official update

- Detects contradiction
- Prioritizes latest authoritative notice
- Applies user category
- Outputs:

> “Answer A is most reliable (78%) because it is based on the official notification dated X, supersedes earlier rules, and applies to your category. Other sources reference outdated or generalized criteria.”

---

## Why This Is a Platform-Level Idea

This system can sit:

- on top of government portals
- inside education platforms
- within enterprise compliance tools
- as an API for other AI systems

It does **not compete directly** with Google, ChatGPT, or news platforms — it **stabilizes** them.

---

## Brutally Honest Boundary

- This system **does not guarantee truth**
- It provides the **best-justified answer at a given time**
- It **must show uncertainty where ambiguity exists**
- In some cases, it may say:

  > “No reliable resolution is possible yet”

That honesty is a **feature**, not a weakness.

---

## 2 a) How is this different from existing ideas

## The Core Difference

**Every existing system either *retrieves*, *generates*, or *labels* information — none of them *resolve contradictions as a first-class function*.**

This is not a small difference. It is a **different problem definition**.

---

## Comparison With Existing Categories (Brutally Honest)

| Existing System | What It Does | Why It Fails in This Problem |
|---|---|---|
| **Search Engines (Google)** | Lists sources | Forces user to decide which source is correct |
| **Chatbots (LLMs)** | Generates a single confident answer | Often hides uncertainty, may hallucinate |
| **Fact-checking sites** | Labels claims as true/false | Too slow, narrow, reactive |
| **Government portals** | Publish official info | Fragmented, outdated, hard to interpret |
| **Social platforms** | Amplify engagement | Reward virality, not correctness |

**What none of them do:**

- Detect *conflicts* explicitly
- Compare *authority vs recency vs applicability*
- Explain *why one answer should be trusted over another*

---

## Architectural Difference (Important for Judges)

Existing systems:

> Input → Retrieve/Generate → Output

**This system:**

> Input → Multi-source ingestion → Contradiction detection → Arbitration → Explainable resolution → Output

This is **decision intelligence**, not content intelligence.

---

# b) How will it actually solve the problem

This is where most ideas collapse. So let’s be precise.

---

## Step-by-Step Problem Solving Mechanism

## Step 1: Structured Multi-Source Ingestion

The system pulls information from:

- official documents
- policy updates
- reputable news
- secondary explainers (tagged as non-authoritative)

All sources are **versioned and timestamped**.

No scraping fantasies — only permitted/public sources.

---

## Step 2: Contradiction Detection (Critical Step)

The system compares claims such as:

- eligibility criteria
- deadlines
- numeric thresholds
- rule applicability

Contradictions are detected when:

- two sources assert mutually exclusive facts
- newer versions override older ones
- generalized advice conflicts with category-specific rules

This is done via **claim extraction + semantic comparison**, not keyword matching.

---

## Step 3: Source Evaluation (Not “Trust Scores”)

Each claim is evaluated on **clear, defensible dimensions**:

- Source authority (official > secondary)
- Recency (latest override wins)
- Scope (general vs specific)
- Applicability (user context)

No fake “AI trust metric.”

Every factor is auditable.

---

## Step 4: Contextual Arbitration

The system does **not** pick a universal truth.

It asks:

- Who is the user?
- Where are they located?
- Which rule applies to *this category*?

Two users may get **different correct answers** to the same question — and that’s realistic.

---

## Step 5: Explainable Resolution Output

Final output includes:

- Selected answer
- Confidence range (not certainty)
- Clear explanation of:
  - why this answer wins
  - why others are less reliable

If resolution is impossible, the system **explicitly says so**.

---

## Why This Solves the Problem (In Reality)

Users don’t need:

- more links
- more opinions
- more confident answers

They need:

> clarity under disagreement

This system replaces **manual comparison + guessing** with **reasoned arbitration**.

---

# c) USP of the Proposed Solution

## USP #1: Contradiction-Aware by Design

Most AI systems break when sources disagree.

This system **expects disagreement** and is built around it.

---

## USP #2: Explainability Is Mandatory, Not Optional

The system is useless without explanation.

Every answer includes:

- reasoning
- evidence hierarchy
- limitations

This aligns with **responsible AI** expectations.

---

## USP #3: Context-Specific Truth (Not Absolute Truth)

It acknowledges a hard reality:

> Truth is often conditional, not universal.

This is rare, honest, and technically difficult.

---

## USP #4: Honest Uncertainty Handling

If evidence is insufficient, the system says:

> “No reliable resolution is possible yet.”

This alone differentiates it from most AI products.

---

### USP #5: Acts as Infrastructure, Not an App

- Can be an API
- Can sit behind portals
- Can stabilize other AI systems

This gives it **long-term relevance**, not hackathon flash.

---

# 3.List of Features Offered by the Solution

---

## 1. Multi-Source Answer Ingestion

**What it does**

- Collects answers/information from **multiple heterogeneous sources** for the same question.
- Sources are tagged as:
  - Official / Primary
  - Secondary / Explanatory
  - User-generated / Opinion

**Why it matters**

- Contradiction cannot be detected unless **multiple viewpoints exist**.
- This is the foundation of the entire system.

**Reality check**

- For hackathon: limit to 3–5 controlled sources (e.g., official PDF + news + blog).

---

## 2. Claim Extraction & Normalization

**What it does**

- Breaks long documents or answers into **atomic claims**:
  - rules
  - dates
  - thresholds
  - eligibility conditions

**Example**

> “Applicants must be 18 years old as of 1 Jan 2024”

Becomes a structured claim

**Why it matters**

- Contradictions exist at the **claim level**, not document level.

---

## 3. Contradiction Detection Engine

**What it does**

- Identifies when two or more claims:
  - logically conflict
  - override one another
  - differ due to versioning or scope

**Types of contradictions detected**

- Temporal (old vs new rule)
- Scope-based (general vs category-specific)
- Numeric (different thresholds)
- Applicability-based (who it applies to)

**Brutal honesty**

- This is **probabilistic**, not perfect.
- The system flags *potential contradictions*, then reasons over them.

---

# 4. Source Authority & Recency Evaluation

**What it does**

- Scores claims based on:
  - source type (official > secondary)
  - last update timestamp
  - version references
  - citation traceability

**Important**

- This is **not a "trust score" fantasy**.
- It is a **transparent weighting system**.

**Why judges like this**

- Auditable
- Explainable
- Responsible

---

# 5. Context-Aware Applicability Engine

**What it does**

- Applies user context such as:
  - location
  - role (student, business, citizen)
  - category (general / reserved / special case)

**Result**

- Two users asking the same question may receive **different correct answers**.

**Why this is powerful**

- Most misinformation spreads because context is ignored.

---

# 6. Truth Arbitration Layer (Core Feature)

**What it does**

- Selects the **best-supported claim set** based on:
  - authority
  - recency
  - applicability
  - internal consistency

**Output**

- One resolved answer
- Not “truth”, but **best-justified position at this moment**

**Failure handling**

- If arbitration confidence is low, the system **refuses to over-answer**.

---

# 7. Confidence & Uncertainty Scoring

**What it does**

- Provides a **confidence range**, not certainty:
  - e.g., 72% confidence

**Why this matters**

- Prevents false confidence
- Signals when users should double-check or wait

**This is rare and valuable**

- Most AI systems hide uncertainty.

---

## 8. Explainable Reasoning Panel

**What it does**

- Shows:
  - why this answer was chosen
  - which sources were deprioritized
  - what evidence overruled what

**Example**

> "This answer is preferred because it is based on the latest official notification dated X, which supersedes earlier rules referenced by other sources."

**Judge appeal**

- Strong alignment with Responsible AI principles.

---

## 9. Source Traceability & Evidence Links

**What it does**

- Every resolved answer links back to:
  - source documents
  - timestamps
  - versions

**Why it matters**

- Builds trust
- Enables verification
- Reduces liability

---

## 10. "No Reliable Resolution" Fallback

**What it does**

- Explicitly states when:

- sources conflict irreconcilably
- no authoritative update exists

**Example**

> “At present, no reliable resolution is possible due to conflicting official statements.”

**This is a strength, not a weakness.**

---

## 11. Domain-Scoped Deployment (Hackathon-Friendly)

**What it does**

- Limits resolution to one domain:
  - exams
  - schemes
  - policies
  - regulations

**Why**

- Keeps the prototype **realistic and complete**, not overpromised.

---

## 12. API-First Design

**What it does**

- Exposes:
  - question input
  - resolved answer
  - explanation
  - confidence score

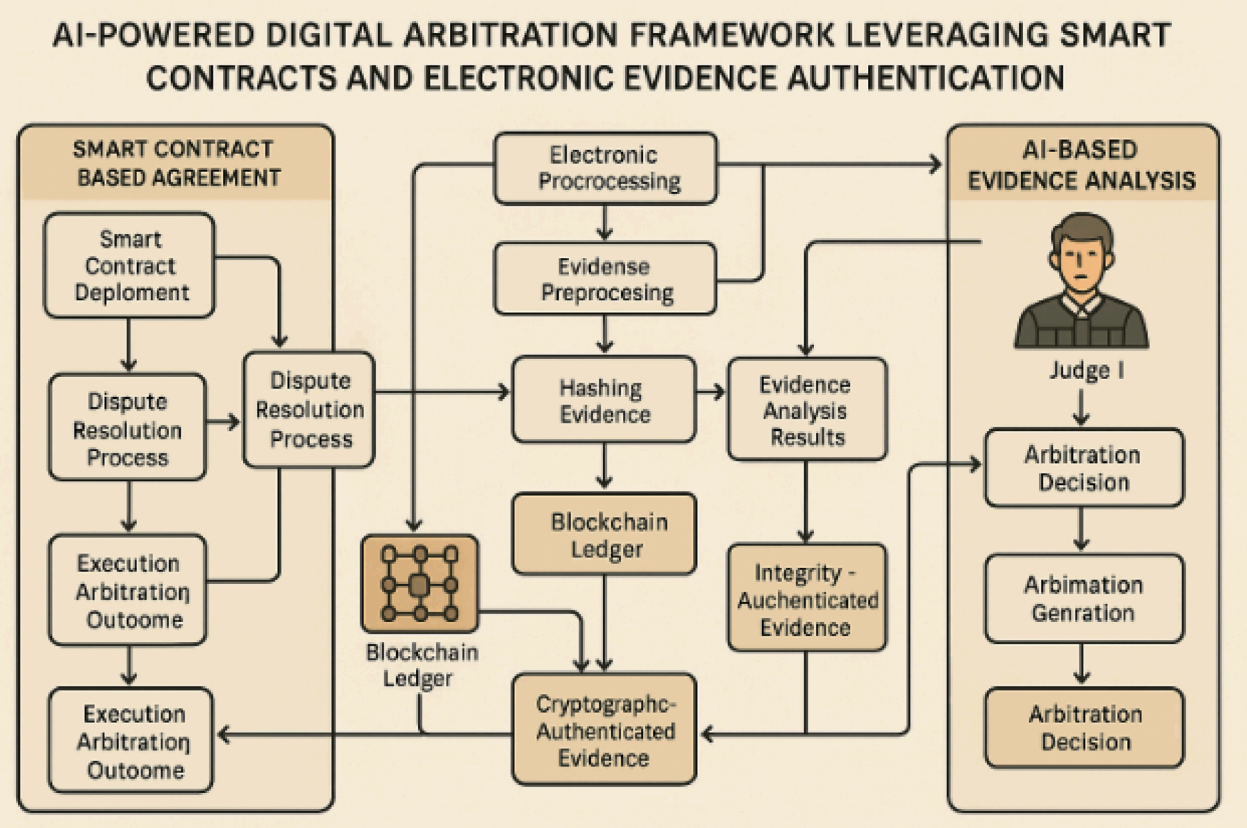
**Why**

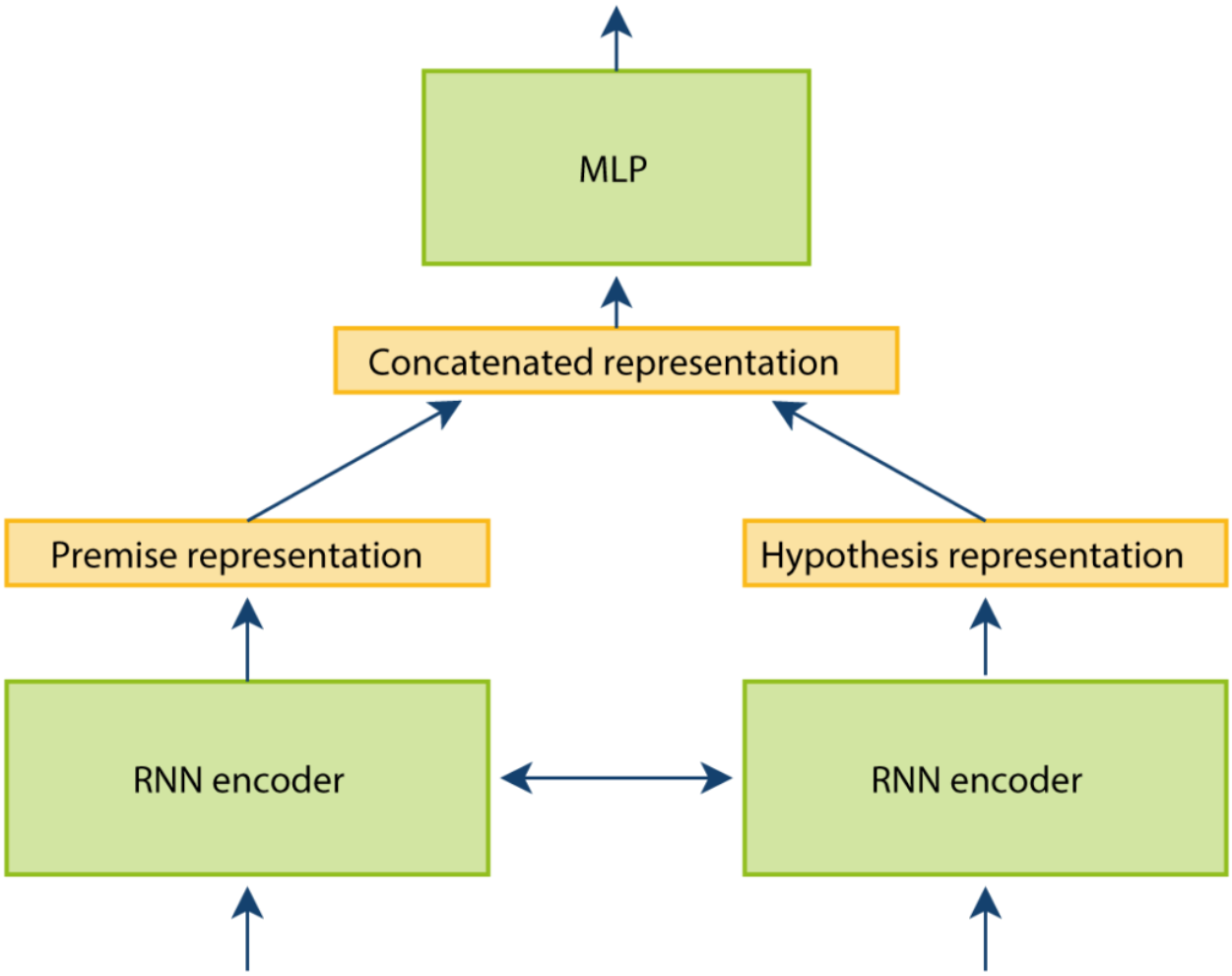
- Shows platform thinking

- Allows reuse across apps

# 4.High-Level Process Flow Diagram

## *(End-to-End Truth Resolution Flow)*






## Flow (Explain This Verbally to Judges)

User Question
↓
Multi-Source Information Fetch
↓
Claim Extraction & Normalization
↓
Contradiction Detection

```
    ↓
Source Authority & Recency Evaluation
    ↓
Context-Aware Arbitration
    ↓
Confidence Scoring
    ↓
Explainable Resolved Answer
```

### Why this flow is IMPORTANT

- Every step **reduces uncertainty**
- AI is used where **human comparison breaks**
- Output is **decision-ready**, not just informative

> Judges will immediately see this is not a chatbot loop.

---

# 2️⃣ Use-Case Diagram

### *(Who uses it and how)*

### Primary Use Cases

### Use Case 1: Citizen / Student / Professional

- Inputs a question
- Receives:
  - resolved answer
  - confidence score
  - explanation
  - source links

### Use Case 2: Platform / Portal (API Consumer)

- Sends query programmatically
- Receives structured response
- Displays answer inside:
  - govt portal
  - education platform
  - enterprise system

### Use Case 3: Admin / Auditor

- Reviews:
  - source hierarchy
  - decision reasoning
  - uncertainty flags

---

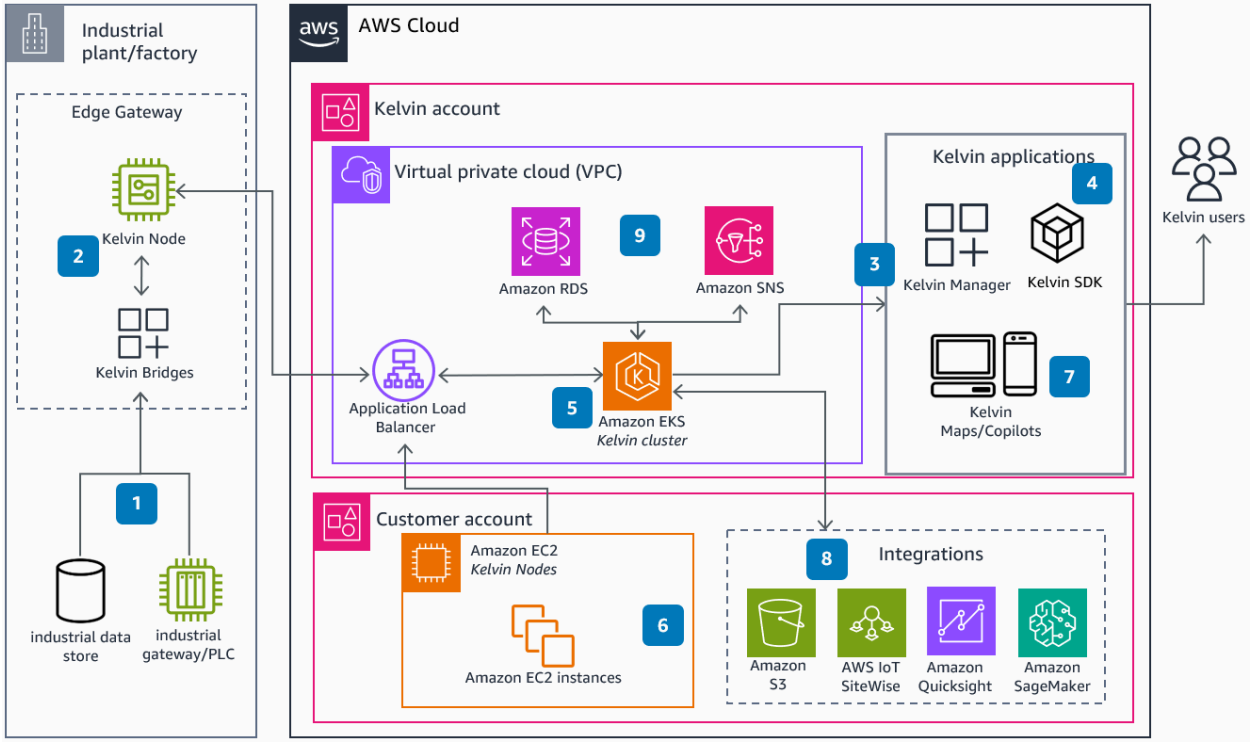
### Why This Use-Case Design Is Strong

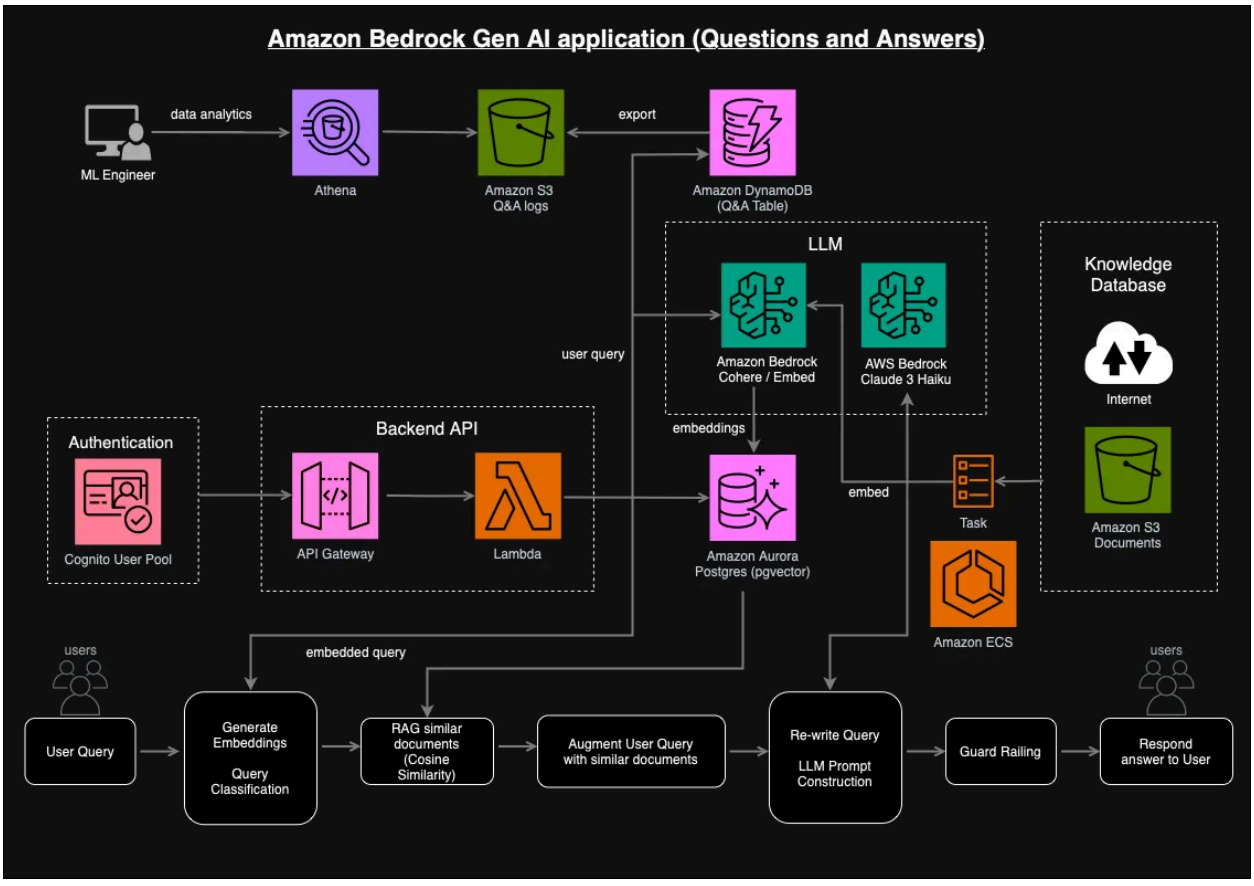
- Shows **multi-stakeholder relevance**
- Emphasizes **API-first infrastructure**
- Signals **governance & responsibility**

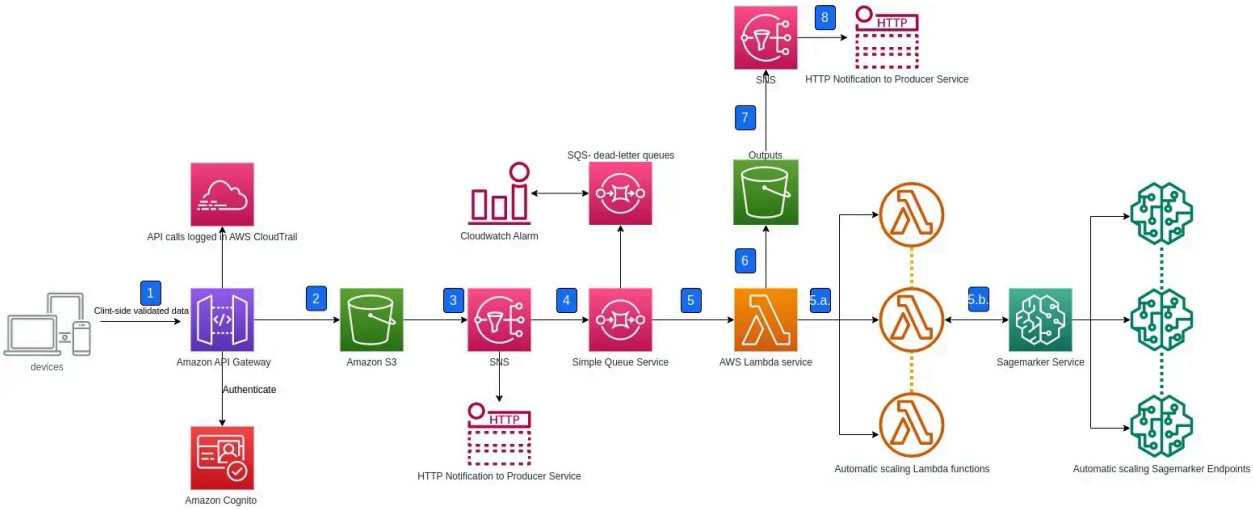
---

# 3️⃣ AWS Architecture Diagram

***(This is where Amazon judges lean forward)***


Industrial plant/factory
Edge Gateway
Kelvin Node
Kelvin Bridges
industrial data store
industrial gateway/PLC
AWS Cloud
Kelvin account
Virtual private cloud (VPC)
Amazon RDS
Amazon SNS
Application Load Balancer
Amazon EKS
Kelvin cluster
Kelvin applications
Kelvin Manager
Kelvin SDK
Kelvin Maps/Copilots
Kelvin users
Customer account
Amazon EC2
Kelvin Nodes
Amazon EC2 instances
Integrations
Amazon S3
AWS IoT SiteWise
Amazon Quicksight
Amazon SageMaker


Amazon Bedrock Gen AI application (Questions and Answers)
ML Engineer
data analytics
Athena
Amazon S3
Q&A logs
export
Amazon DynamoDB
(Q&A Table)
LLM
Amazon Bedrock
Cohere / Embed
AWS Bedrock
Claude 3 Haiku
Knowledge Database
Internet
Amazon S3
Documents
user query
embeddings
embed
Task
Amazon ECS
Authentication
Cognito User Pool
Backend API
API Gateway
Lambda
Amazon Aurora
Postgres (pgvector)
users
embedded query
User Query
Generate Embeddings
Query Classification
RAG similar documents (Cosine Similarity)
Augment User Query with similar documents
Re-write Query
LLM Prompt Construction
Guard Railing
Respond answer to User




# Architecture Breakdown (Layer by Layer)

## 🔷 1. User / Client Layer

- Web app / Mobile app / Portal
- Sends question via HTTPS

**AWS Services**

- Amazon API Gateway
- AWS Cognito (optional auth)

---

## 🔷 2. Ingestion & Source Layer

- Pulls data from:
  - Official documents (PDFs)
  - Trusted news APIs
  - Pre-registered sources

**AWS Services**

- Amazon S3 (document storage)
- AWS Lambda (fetch & parsing)
- Amazon Textract (if PDFs involved)

## 🔷 3. Claim Extraction & Understanding

- Breaks content into atomic claims
- Normalizes entities (dates, rules, thresholds)

**AWS Services**

- Amazon Bedrock (LLM-based extraction)
- Amazon Comprehend (entity detection)

---

## 🔷 4. Contradiction Detection Engine

- Compares claims across sources
- Flags conflicts, overrides, inconsistencies

**AWS Services**

- AWS Lambda
- Amazon OpenSearch (semantic comparison)

---

## 🔷 5. Arbitration & Reasoning Layer (Core Intelligence)

- Evaluates:
  - authority
  - recency
  - applicability
- Selects best-supported claim set

**AWS Services**

- Amazon Bedrock (reasoning & arbitration)
- AWS Step Functions (decision orchestration)

---

## 🔷 6. Confidence & Explainability Layer

- Assigns confidence range

- Generates reasoning trace

**AWS Services**

- Amazon Bedrock
- Amazon DynamoDB (decision logs)

---

### 🔷 7. Response Layer

- Returns:
  - resolved answer
  - confidence score
  - explanation
  - evidence links

**AWS Services**

- API Gateway
- AWS Lambda

---

# 4️⃣ One-Slide Version

You can compress everything into:

```
User Question
↓
Multiple Conflicting Sources
↓
AI Detects Contradictions
↓
AI Weighs Authority + Recency + Context
↓
Single Explainable Answer + Confidence
```

---

# 5.DESIGN PRINCIPLES

Before the wireframes, these principles matter to judges:

- **Clarity > Beauty**
- **Explainability > Speed**
- **Confidence-with-humility > Bold claims**
- **Infrastructure UI, not consumer gimmicks**

This UI feels like:

> "A system you trust with decisions."

---

# 1️⃣ Screen 1: Question Input & Context Capture

## Purpose

Allow the user to ask a question **once**, while capturing **context silently**.

---

## Wireframe (Low-Fidelity)

```
┌──────────────────────────────────────────────┐
│ AI Truth Resolution Engine                   │
├──────────────────────────────────────────────┤
│ Ask your question                            │
│ [_____________________________________]      │
│                                              │
│ Context (optional but recommended):          │
│ [📍 Location: Maharashtra ▾ ]                │
│ [👤 Category: Student ▾ ]                    │
│ [📄 Domain: Education ▾ ]                    │
│                                              │
```

```
| [ Resolve Conflicting Information ]  |
└──────────────────────────────────────┘
```

## Why this design works

- One **clear call to action**
- Context is **explicit**, not inferred blindly
- Avoids hallucination by asking *just enough*

## How to explain to judges

> “We don’t assume context. Users can explicitly tell the system who they are so the resolution is accurate.”

# 2️⃣ Screen 2: Source Landscape & Contradiction View

## Purpose

Show **why the problem exists** before showing the answer.

## Wireframe

```
┌──────────────────────────────────────┐
| Detected Information Landscape       |
├──────────────────────────────────────┤
| ❌ Conflicting Answers Found (4)     |
|                                      |
| Source A (Official)                  |
| → Eligibility age: 18+               |
| → Notification: Jan 2024             |
```

```
│                                  │
│ Source B (Blog)                  │
│ → Eligibility age: 21+           │
│ → Updated: Oct 2023              │
│                                  │
│ Source C (Video)                 │
│ → Eligibility unclear            │
│                                  │
│ Source D (Old Notice)            │
│ → Eligibility age: 17+           │
└──────────────────────────────────┘
```

---

## Why this is powerful

- Makes **contradiction visible**
- Proves the problem is real
- Judges *feel* the chaos immediately

---

## How to explain to judges

> "Before resolving, we first expose the contradiction so users understand why blind trust fails."

---

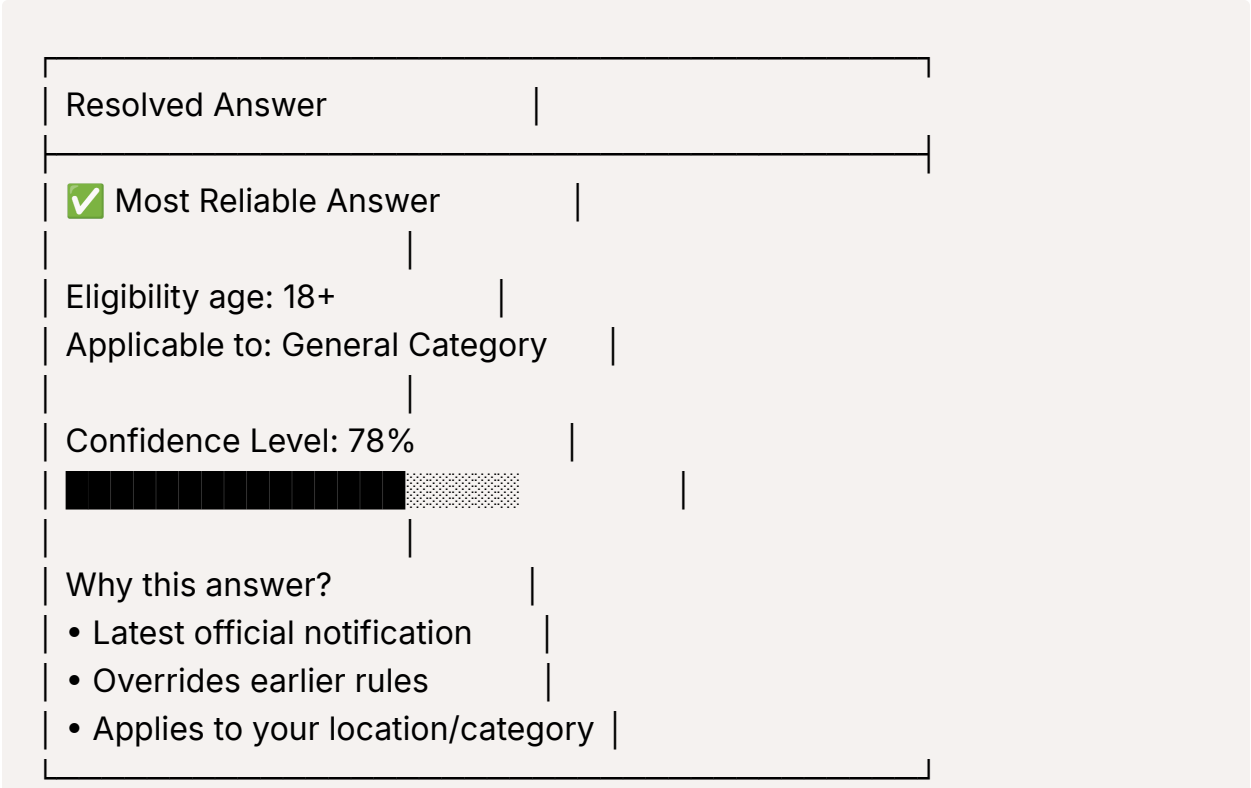
# 3️⃣ Screen 3: Truth Resolution & Confidence Output (CORE SCREEN)

## Purpose

Deliver the **resolved answer**, not as a claim, but as a **justified decision**.

---

## Wireframe




## Why this is different

- Confidence ≠ certainty
- Explanation is **mandatory**
- UI discourages blind trust

## How to explain to judges

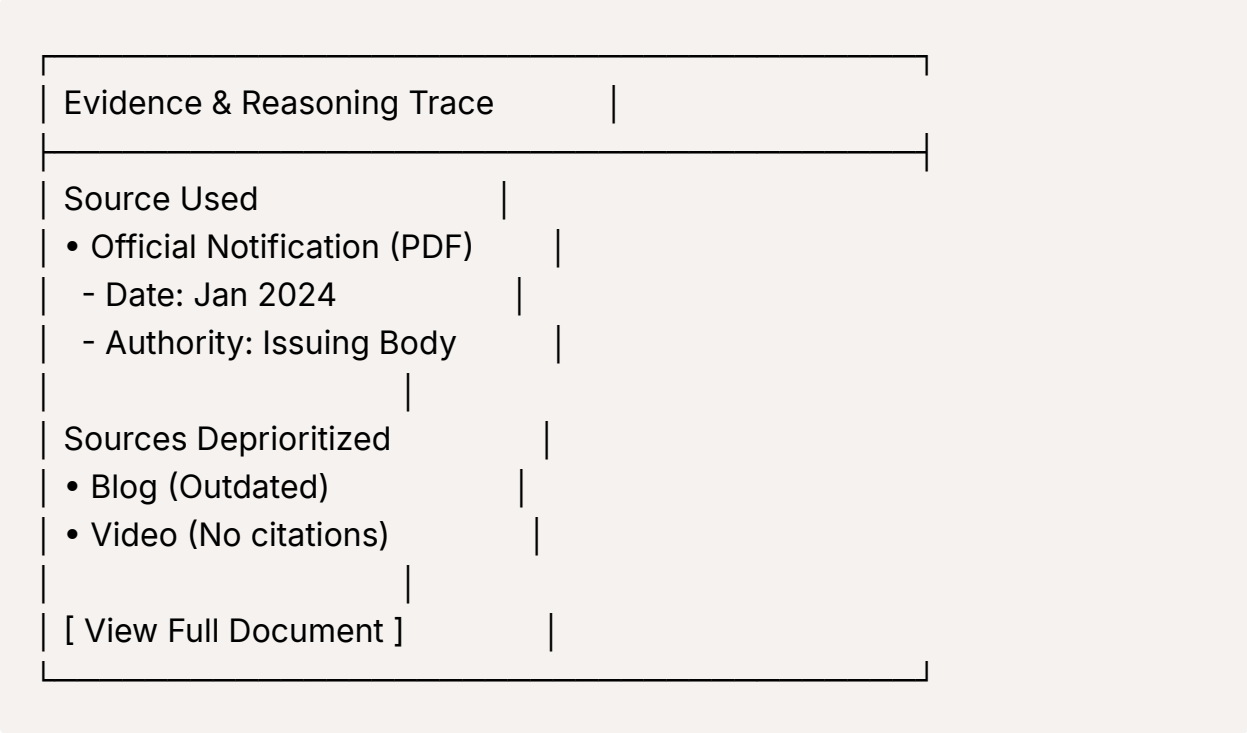
> “We show confidence ranges and reasoning because responsible AI should never pretend to be infallible.”

# 4️⃣ Screen 4: Evidence & Audit Trail (Trust Builder)

## Purpose

Allow verification and accountability.

## Wireframe




## Why this matters

- Enables **human verification**
- Reduces legal and ethical risk
- Judges see **auditability**

## How to explain to judges

> “This system never hides sources. Every decision can be audited.”

# 5️⃣ “No Reliable Resolution”

*(Judges LOVE this honesty)*

```
┌──────────────────────────────────────────┐
│ ⚠ No Reliable Resolution Available       │
├──────────────────────────────────────────┤
│ Reason:                                  │
│ • Conflicting official notices           │
│ • No newer clarification exists          │
│                                          │
│ Recommendation:                          │
│ • Wait for official update               │
│ • Contact issuing authority              │
└──────────────────────────────────────────┘
```

This screen alone separates you from 90% of teams.

# 6.Architecture Diagram — AI Truth Resolution Engine

## 1️⃣ High-Level Architecture (Conceptual)

```
[ Client / UI / API Consumer ]
            |
            ▼
      Amazon API Gateway
            |
            ▼
       AWS Lambda
            |
            ▼
┌──────────────────────────────────────────┐
│   Truth Resolution Pipeline              │
└──────────────────────────────────────────┘
```

```
        |
        ▼
Explainable Resolved Answer
```

This looks simple **by design**.

The intelligence lives inside the pipeline, not the UI.

---

# 2️⃣ Architecture Breakdown (Layer by Layer)

---

## 🔷 Layer 1: Client / Access Layer

### What it does

- Accepts user questions
- Collects explicit context (location, category, domain)
- Displays resolved output

### Examples

- Web app
- Government portal
- Enterprise dashboard
- API consumer

### AWS Services

- **Amazon API Gateway**
- **AWS Cognito** (optional, for auth)

### Why this matters

- Keeps UI thin
- Enables API-first deployment
- Supports multiple consumers

## 🔷 Layer 2: Request Orchestration Layer

### What it does

- Validates input
- Initiates truth resolution workflow
- Handles retries and failures

### AWS Services

- **AWS Lambda**
- **AWS Step Functions**

### Why Step Functions

- Clear state transitions
- Judge-friendly visual workflows
- Easier auditability than monolithic logic

---

## 🔷 Layer 3: Source Ingestion & Storage Layer

### What it does

- Fetches information from **pre-approved sources**
- Stores documents with versioning

### Data Types

- PDFs
- Web text
- Structured notices

### AWS Services

- **Amazon S3** (document storage & versioning)

- **AWS Lambda** (fetch/parsing)
- **Amazon Textract** (for PDFs, if required)

### Brutal honesty

- This system **does not crawl the entire internet**
- Sources are curated → this increases reliability

---

## 🔷 Layer 4: Claim Extraction & Normalization Layer

### What it does

- Converts raw documents into **atomic claims**
- Normalizes:
  - dates
  - rules
  - eligibility conditions
  - numeric thresholds

### AWS Services

- **Amazon Bedrock** (LLM-based extraction)
- **Amazon Comprehend** (entities & relationships)

### Why AI is necessary here

Rule-based NLP fails when:

- language varies
- documents are unstructured
- phrasing is inconsistent

---

## 🔷 Layer 5: Contradiction Detection Layer

### What it does

- Compares claims across sources
- Flags:
  - conflicts
  - overrides
  - outdated rules
  - ambiguous interpretations

### AWS Services

- **Amazon OpenSearch** (semantic similarity & comparison)
- **AWS Lambda**

### Important limitation (honest)

- Contradictions are detected **probabilistically**
- Final decision is handled in the next layer

---

## 🔷 Layer 6: Arbitration & Reasoning Layer (CORE)

### What it does

This is the **heart of the system**.

It:

- Evaluates authority
- Checks recency
- Applies user context
- Resolves conflicts
- Decides whether resolution is even possible

### AWS Services

- **Amazon Bedrock** (reasoning & justification)
- **AWS Step Functions** (decision flow control)

### Why Bedrock (and not “LLM magic”)

- Controlled models
- Enterprise-grade
- Responsible AI alignment

---

## 🔷 Layer 7: Confidence & Explainability Layer

### What it does

- Assigns confidence range
- Generates reasoning trace
- Lists deprioritized sources

### AWS Services

- **Amazon Bedrock**
- **Amazon DynamoDB** (decision logs & audit data)

### Why this layer exists

- Prevents overconfidence
- Enables audits
- Builds institutional trust

---

## 🔷 Layer 8: Response & Audit Layer

### What it does

- Returns structured output:
  - resolved answer

- confidence score
- explanation
- source links

### AWS Services

- **API Gateway**
- **AWS Lambda**
- **Amazon CloudWatch** (logs & monitoring)

---

## 3️⃣ Architecture Principles (Judges Care About This)

### ✔️ What This Architecture Gets Right

- Serverless → scalable & hackathon-friendly
- Explainability baked in
- Clear separation of concerns
- AI used only where necessary

### ❌ What This Architecture Does NOT Pretend

- Not real-time for all domains
- Not omniscient
- Not hallucination-free

This honesty **increases credibility**, not reduces it.

---

## 4️⃣ One-Slide Architecture Summary (For Presentation)

If you need a **single slide**, use this:

```
User Query
↓
Multiple Trusted Sources (S3)
```

```
↓
Claim Extraction (Bedrock)
↓
Contradiction Detection (OpenSearch)
↓
AI Arbitration + Context (Bedrock)
↓
Explainable Answer + Confidence
```

Say this line aloud:

> “We don’t generate answers first. We resolve disagreement first.”

That line sticks.

---

# 7.Technology stack

## 1️⃣ Core Cloud Platform

### ☁️ Amazon Web Services (AWS)

**Why AWS**

- Native support for **foundation models (Bedrock)**
- Serverless-first (ideal for hackathon + scale)
- Strong emphasis on **responsible AI, auditability**
- Judges explicitly evaluate *effective AWS usage*

---

## 2️⃣ AI / ML Technologies (The Brain)

### 🧠 Amazon Bedrock

**Used for:**

- Claim extraction from unstructured text

- Semantic contradiction reasoning
- Arbitration & justification generation
- Confidence explanation (not answer hallucination)

**Why Bedrock (brutally honest)**

- Managed foundation models
- Better control than raw open-source LLMs
- Enterprise-grade logging & safety
- Judges prefer **Bedrock over "random LLM APIs"**

**What we do NOT claim**

- Not "100% accurate"
- Not autonomous truth
- Used strictly for *reasoning over extracted claims*

---

## 🧩 Amazon Comprehend

**Used for:**

- Named entity recognition (dates, rules, organizations)
- Language normalization
- Relationship extraction

**Why needed**

- Improves structure before LLM reasoning
- Reduces hallucination risk
- Cheap and fast for preprocessing

---

# 3️⃣ Data Ingestion & Storage Layer

## 🗄️ Amazon S3

**Used for:**

- Storing source documents (PDFs, HTML, text)
- Versioning of documents
- Audit-safe immutable storage

**Why S3**

- Cheap, scalable, reliable
- Built-in versioning helps detect outdated sources
- Judges love seeing S3 used *correctly*

---

### 📄 Amazon Textract

*(Optional, domain-dependent)*

**Used for:**

- Extracting text from official PDFs and scanned notices

**Why**

- Many government documents are PDFs
- Manual parsing is unreliable
- Textract keeps pipeline realistic

---

## 4️⃣ Search & Contradiction Detection Layer

### 🔍 Amazon OpenSearch Service

**Used for:**

- Semantic indexing of extracted claims
- Similarity comparison across sources
- Fast contradiction candidate detection

**Why OpenSearch**

- Designed for text-heavy workloads
- Works well with embeddings

- Better than SQL for semantic comparisons

**What it is NOT used for**

- Final decision making
- “Truth scoring”

It only **narrows down conflicting claims**.

---

# 5️⃣ Workflow Orchestration & Decision Control

## 🔁 AWS Step Functions

**Used for:**

- Orchestrating multi-step resolution pipeline
- Enforcing decision order:

  ingestion → extraction → contradiction → arbitration

**Why this matters**

- Makes logic **explicit and reviewable**
- Easier to debug than monolithic Lambdas
- Strong signal of engineering maturity

---

## ⚙️ AWS Lambda

**Used for:**

- Lightweight processing
- API handling
- Data transformation
- Glue code between services

**Why Lambda**

- Serverless
- Cost-effective

- Hackathon-friendly

---

## 6️⃣ API & Access Layer

### 🌐 Amazon API Gateway

**Used for:**

- Exposing Truth Resolution API
- Rate limiting
- Secure access for UI and external systems

**Why**

- Clean separation of frontend & backend
- Makes solution platform-ready

---

### 🔐 Amazon Cognito *(Optional)*

**Used for:**

- User authentication
- Role-based access (user vs admin vs auditor)

**Why optional**

- Not critical for hackathon demo
- But shows enterprise readiness

---

## 7️⃣ Explainability, Logging & Audit (CRITICAL)

### 📊 Amazon DynamoDB

**Used for:**

- Storing resolution logs
- Confidence scores
- Reasoning traces

**Why DynamoDB**

- Fast reads
- Serverless
- Perfect for audit records

---

### 📈 Amazon CloudWatch

**Used for:**

- Monitoring
- Logging
- Error tracing
- Transparency for judges

---

## 8️⃣ Frontend & Visualization (Minimal but Serious)

### 🖥️ Web Frontend

**Technologies**

- React / Next.js
- Simple UI (no animations, no gimmicks)

**Why minimal UI**

- Judges care about **reasoning**
- Overdesigned UI hurts credibility

---

## 9️⃣ Developer & AI Workflow Tools

### 🧪 AI-Assisted Development

- Prompt versioning for Bedrock
- Prompt templates (claim extraction vs arbitration)
- Evaluation datasets (small, controlled)

**Why mention this**

- Hackathon guidelines encourage AI-assisted workflows
- Shows maturity without overclaiming

---

## 1️⃣0️⃣ What We Intentionally Do NOT Use (Very Important)

This section increases trust.

❌ No social media scraping

❌ No private or sensitive data

❌ No black-box "trust score"

❌ No unsupervised web crawling

❌ No claim of "absolute truth"

❌ No self-learning from user behavior

---

## 1️⃣1️⃣ Technology Stack Summary Table (Slide-Ready)

| Layer | Technology |
|---|---|
| AI Reasoning | Amazon Bedrock |
| NLP Structuring | Amazon Comprehend |
| Document Storage | Amazon S3 |
| PDF Parsing | Amazon Textract |
| Semantic Search | Amazon OpenSearch |
| Workflow | AWS Step Functions |
| Compute | AWS Lambda |
| API | Amazon API Gateway |
| Audit Logs | Amazon DynamoDB |
| Monitoring | Amazon CloudWatch |
| Frontend | React / Next.js |

---

# 8.Estimated Implementation Cost

## Costing Assumptions (Very Important)

Before numbers, judges care about assumptions.

**Scope assumed:**

- Single domain (e.g., exams / policies)
- 3–5 trusted sources
- English only (initially)
- ~1,000–2,000 queries/month (hackathon demo + light testing)
- Serverless, on-demand usage
- No real-time streaming

---

# 1️⃣ Hackathon MVP Cost (Most Important)

### 🎯 Goal

- Working prototype
- Demonstrates contradiction detection + arbitration + explainability
- Not production scale

### 🕒 Duration

- 2–4 weeks

---

## AWS Cost Breakdown (Monthly)

### 🧠 Amazon Bedrock (LLM usage)

**Used for:**

- Claim extraction
- Arbitration

- Explanation generation

**Estimate**

- ~2–3 LLM calls per query
- Short prompts + constrained outputs

👉 **₹2,000 – ₹4,000 / month**

(You can keep this low by:

- limiting context
- caching results
- restricting domains)

---

## 🔍 Amazon OpenSearch

**Used for:**

- Semantic comparison
- Contradiction candidate detection

**Smallest viable cluster**

👉 **₹2,000 – ₹3,000 / month**

(For hackathon, a **single-node dev cluster** is acceptable.)

---

## 🗄️ Amazon S3

**Used for:**

- Source documents
- Versioning
- Logs

👉 **₹200 – ₹500 / month**

(S3 is cheap; this will not be your problem.)

---

## ⚙️ AWS Lambda

**Used for:**

- Orchestration
- Parsing
- API handling

👉 **₹300 – ₹700 / month**

(Serverless = minimal idle cost.)

---

### 🔁 AWS Step Functions

**Used for:**

- Pipeline orchestration

👉 **₹200 – ₹400 / month**

---

### 🌐 API Gateway

👉 **₹100 – ₹300 / month**

---

### 📊 DynamoDB + CloudWatch

👉 **₹300 – ₹600 / month**

---

## ✅ Total Hackathon MVP Cost

### 💰 ₹5,000 – ₹10,000 per month

(≈ **$60 – $120/month**)

> This is a very defensible number in front of judges.

---

# 2️⃣ Pilot Deployment Cost (Post-Hackathon)

### 🎯 Goal

- Limited real users (e.g., college / department / portal)

- Multi-language (2–3 languages)
- Higher query volume

### Usage Assumption

- ~10,000 queries/month

---

## Cost Estimate

| Component | Monthly Cost |
|---|---|
| Amazon Bedrock | ₹15,000 – ₹25,000 |
| OpenSearch (scaled) | ₹6,000 – ₹10,000 |
| Lambda + Step Functions | ₹3,000 – ₹5,000 |
| Storage & Logs | ₹1,000 – ₹2,000 |

### 💰 Total: ₹25,000 – ₹40,000 / month

---

# 3️⃣ Early-Scale / Production (Reality Check)

### 🎯 Goal

- Multiple domains
- API consumers
- Thousands of daily queries

### Usage

- ~100,000 queries/month

---

## Cost Reality (No Sugarcoating)

### 💰 ₹2 – ₹4 per resolved query

Breakdown:

- LLM reasoning (dominant cost)
- Search & orchestration
- Logging & audit

👉 **₹2,00,000 – ₹4,00,000 / month**

This is **normal** for reasoning-heavy AI systems.

---

# 4️⃣ What Drives Cost Up (Honest List)

- Long documents (PDF-heavy domains)
- Multi-language reasoning
- Overuse of LLM calls
- Poor caching
- Too many sources per query

---

# 5️⃣ Cost Control Strategies (Judges Will Like This)

## ✅ Domain scoping

Resolve **only one domain at a time**

## ✅ Aggressive caching

Most questions repeat

## ✅ Claim-level reuse

Claims don’t change often

## ✅ “No resolution” short-circuit

Don’t invoke full pipeline unnecessarily

## ✅ Human-in-the-loop (optional later)

For high-risk domains

---

## 6️⃣ What We Are NOT Claiming (Important)

❌ “Free AI”

❌ “Costs ₹0 at scale”

❌ “Unlimited users for cheap”

This honesty **increases trust**.

---

# 9.Alignment With AI for Bharat Hackathon Requirements

*(Mapped explicitly to evaluation criteria & guidelines)*

---

## 1️⃣ Track Alignment (Mandatory)

### Selected Track

**AI for Communities, Access & Public Impact**

### Justification (Clear & Logical)

- The solution directly addresses **information access, clarity, and trust**, which are foundational to:
  - public services
  - education systems
  - civic decision-making
- It improves **access to reliable information**, not by adding more content, but by **resolving contradictions** that block citizens from acting.

> This is a public-impact infrastructure problem, not a consumer productivity tool.

---

## 2️⃣ Meaningful Use of AI (Not Rule-Based)

### Requirement

> "Solutions should demonstrate meaningful use of AI, not rule-based logic alone."

### Compliance Explanation

AI is **essential**, not decorative, in the following places:

| Problem Component | Why Rules Fail | Why AI Is Required |
|---|---|---|
| Claim extraction | Language varies wildly | LLM-based understanding |
| Contradiction detection | Semantic conflicts | Embedding + reasoning |
| Arbitration | Multi-factor trade-offs | Probabilistic reasoning |
| Explainability | Natural language justification | LLM explanation synthesis |

✔️ No single rule engine can handle these simultaneously.

---

## 3️⃣ "Why AI?" (Explicit Answer)

### Hackathon Requirement

> Teams should clearly explain why AI is needed

### Clear Statement

> "The problem is not retrieving information, but resolving disagreement between multiple valid-looking sources. This requires semantic understanding, contextual reasoning, and uncertainty handling — capabilities that only modern AI models can provide."

This directly answers the **Why AI** question judges ask.

---

## 4️⃣ AI Workflow Usage (Encouraged)

### Compliance

The solution uses AI in **both development and runtime**, as encouraged:

- AI-assisted prompt engineering for:
  - claim extraction
  - contradiction reasoning
- Prompt versioning and evaluation
- Controlled AI inference using **Amazon Bedrock**

✔️ This shows **AI-assisted development workflow**, not just AI inference.

---

# 5️⃣ Responsible AI Design (Critical)

## Hackathon Emphasis

> "Emphasis on clarity, usability, and responsible design"

## How the Solution Complies

| Responsible AI Principle | How It's Implemented |
|---|---|
| Explainability | Mandatory reasoning output |
| Transparency | Source traceability |
| Uncertainty | Confidence ranges |
| Safety | Refusal to resolve when evidence insufficient |
| Accountability | Audit logs |

> The system explicitly avoids hallucinated certainty.

This is **rare** and highly valued.

---

# 6️⃣ Impact & Relevance (India / Bharat Context)

## Requirement

> "Potential for real-world impact in India"

### Real Bharat Relevance

India faces:

- multiple authorities
- frequent policy updates
- language & interpretation gaps
- WhatsApp-driven misinformation

This solution:

- reduces dependence on social forwarding
- improves trust in official systems
- supports multi-lingual expansion (future scope)

✔️ Impact is **systemic**, not niche.

---

## 7️⃣ Technical Excellence (AWS Usage – 30%)

### Requirement

> "Effective and innovative use of AWS services"

### Explicit AWS Mapping (Judges Look for This)

| AWS Service | Purpose |
|---|---|
| Amazon Bedrock | AI reasoning & arbitration |
| Amazon OpenSearch | Semantic contradiction detection |
| AWS Step Functions | Explainable decision workflows |
| AWS Lambda | Serverless compute |
| Amazon S3 | Versioned document storage |
| Amazon DynamoDB | Audit logs |
| Amazon API Gateway | Platform access |
| Amazon CloudWatch | Monitoring |

✔️ No unused services

✔️ No over-engineering

✔️ Clear reasoning for each service

---

## 8️⃣ Feasibility & Completeness (Prototype-Ready)

### Hackathon Expectation

> “Functionality and polish of the prototype”

### MVP Scope (Explicitly Defined)

- Single domain (e.g., exam eligibility or public notice)
- 3–5 trusted sources
- English only
- Web UI + API response
- Full end-to-end resolution flow

✔️ Achievable in hackathon timeframe

✔️ Demonstrable live

✔️ No unrealistic promises

---

## 9️⃣ Cost & Sustainability (Business Feasibility – 20%)

### Compliance

- Hackathon MVP cost: **₹5k–₹10k/month**
- Query-based scaling model
- API-first (can be offered as a service)

This supports:

- institutional adoption
- government / platform deployment
- long-term sustainability

---

## 🔟 Completeness & Presentation (15%)

### How This Idea Scores High

- Clear problem framing
- Strong visuals (contradiction → resolution)
- Explainable AI demo
- Honest limitations

Judges prefer:

> "This team understands what their AI can and cannot do."

---

## One-Paragraph Submission-Ready Summary

> Our solution addresses a critical public-impact problem: contradictory information across official, media, and digital sources that prevents citizens from making confident decisions. Instead of generating answers, we built an AI Truth Resolution Engine that detects conflicting claims, evaluates authority and recency, applies user context, and produces a single explainable, confidence-ranked answer. Built using AWS Bedrock, OpenSearch, Step Functions, and serverless services, the system emphasizes responsible AI through transparency, uncertainty handling, and auditability. The prototype focuses on a single domain, is cost-efficient, and demonstrates strong real-world relevance for India.

---

## 10.Problem Statement

In India's rapidly digitizing ecosystem, citizens, students, professionals, and businesses increasingly rely on online information to make critical decisions related to education, government services, compliance, employment, and public policy. However, the same question often yields **multiple, conflicting answers** across official notifications, websites, media articles, social platforms, and explanatory content.

This contradiction-driven information environment creates **confusion, mistrust, and decision paralysis**, forcing users to manually compare sources, guess authority, or rely on social consensus rather than verified clarity. Existing systems—such as search engines, chatbots, and portals—either retrieve links, generate confident responses, or publish fragmented updates, but **do not resolve conflicts between sources**, nor explain which information should be trusted, for whom, and why.

As a result, people make incorrect or delayed decisions, benefits and services remain underutilized, misinformation spreads faster than corrections, and trust in digital systems erodes. There is currently **no scalable, context-aware system** that can detect contradictory information, evaluate source authority and recency, apply user-specific context, and provide a **single, explainable, confidence-ranked resolution** to support informed decision-making.

---

## Core Problem (One Line)

> The absence of an intelligent, explainable mechanism to resolve contradictory information prevents individuals and institutions from making confident, timely, and reliable decisions in critical public and professional domains.

---